

H1. 四种排序算法及其操作分析

- 计数排序
- 选择排序
- 冒泡排序
- 插入排序
- **讲解**：思想→示例→代码→分析
- **共识**：所谓排序是指将一组无序元素按照从小到大的次序重新排列的过程，其关键操作是**元素的比较**



计数排序：思想

- 先求得元素在序列中的名次，具体过程是
 - 对于每一个元素，将它与其左边的所有元素比较
 - 谁大，谁的名次就加1
 - 直至结束
- 然后将元素调整到与其名次对应的位置上



计数排序：示例

a	26	33	35	29	19	12	22
---	----	----	----	----	----	----	----

r	3	5	6	4	1	0	2
---	---	---	---	---	---	---	---

u	12	19	22	26	29	33	35
---	----	----	----	----	----	----	----



计数排序：代码和分析

```
template<class T>
void Rank(T a[], int n, int r[]){
    for(int i=0; i<n; i++)
        r[i]=0;
    for (int i = 1; i < n; i++)
        for(int j=0; j<i; j++)
            if (a[j] <= a[i])
                r[i]++;
            else r[j]++;
}
```

循环 $n(n-1)/2$ 次

```
template<class T>
void Rearrange(T a[], int n, int r[]){
    T *u=new T[n];
    for(int i=0; i<n; i++)
        u[r[i]]=a[i];
    for(int i=0; i<n; i++)
        a[i]=u[i];
    delete []u;
}
```

结论：比较次数 $(n-1)n/2$
移动次数 $2n$ ，占用了额外空间！



计数排序改进思想

a	12	19	22	26	29	33	35
---	----	----	----	----	----	----	----

r	0	1	2	3	4	5	6
---	---	---	---	---	---	---	---

○ $r[i] == i$?

- Y: 很好, $a[i]$ 就应该是第 i 个, 它已经在正确位置上了, 什么也不用做, 继续考虑 $a[i+1]$
- N: $a[i]$ 的位置不对, 怎么办?
 - 谁应该在第 i 位我们不知道
 - 但, 我们知道 $a[i]$ 应该在哪—— $r[i]$, 将它与 $a[r[i]]$ 交换, $a[i]$ 即到达正确位置, 新交换来的应该在第 i 位吗? 继续



改进计数排序

```
template<class T>
```

```
void Rearrange(T a[], int n, int r[]){
```

```
    T *u=new T[n];
```

```
    for(int i=0; i<n; i++)
```

```
        u[r[i]]=a[i];
```

```
    for(int i=0; i<n; i++)
```

```
        a[i]=u[i];
```

```
    delete []u;
```

```
}
```

```
template<class T>
```

```
void Rearrange(T a[], int n, int r[]){
```

```
    for(int i=0; i<n; i++)
```

```
        while(r[i]!=i){
```

```
            int t=r[i];
```

```
            Swap(a[i],a[t]);
```

```
            Swap(r[i],r[t]);
```

```
        }
```

**结论：移动次数可能增加
占用空间减少！**



选择排序：思想

- 首先找出最小（大）元素，把它移动到 $a[0]$ ($a[n-1]$)；
- 然后在余下的 $n-1$ 个元素中寻找最小（大）的元素并把它移动到 $a[1]$ ($a[n-2]$)；
- 如此进行下去，直至全部排完。

$$A_0 \leq A_1 \leq \cdots A_{i-1} \mid A_i, \cdots A_{\min}, \cdots, A_{n-1}$$



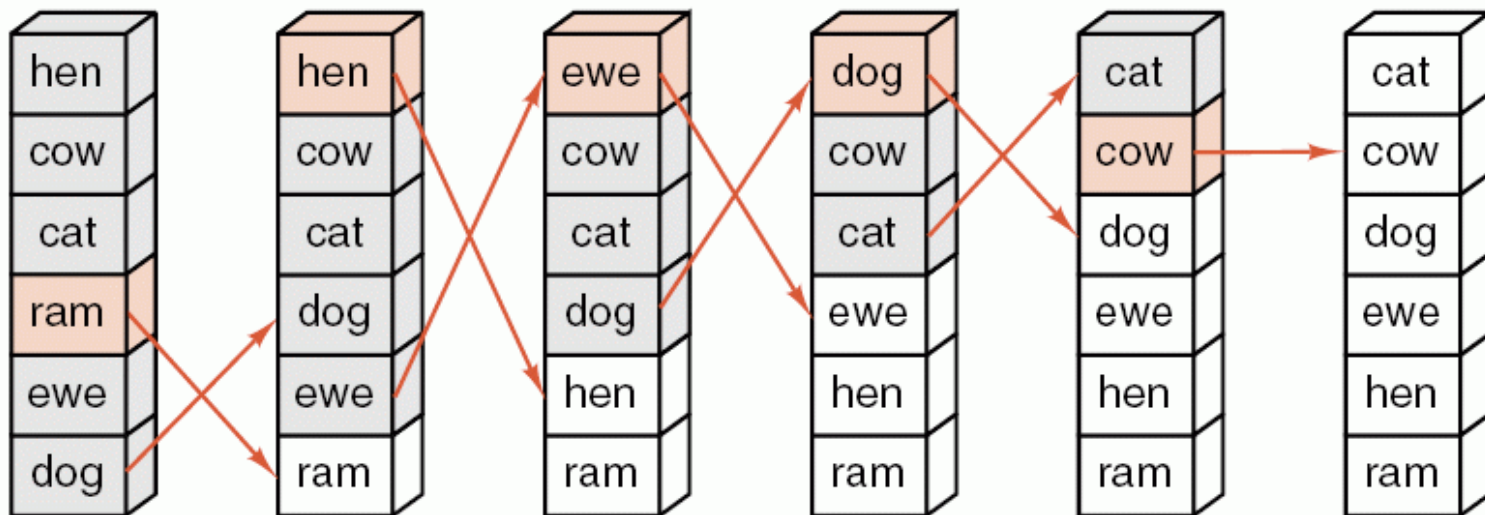

选择排序：示例

	89	45	68	90	29	34	17
17		45	68	90	29	34	89
17	29		68	90	45	34	89
17	29	34		90	45	68	89
17	29	34	45		90	68	89
17	29	34	45	68		90	89
17	29	34	45	68	89		90



选择排序： 示例2

Initial order



Sorted

Colored box denotes largest unsorted key.
Gray boxes denote other unsorted keys.



选择排序：代码和分析

```
template<class T>
```

```
int Max(T a[], int n)
```

```
{
```

```
    int pos = 0;
```

```
    for (int i = 1; i < n; i++)
```

```
        if (a[pos] < a[i])
```

```
            pos = i;
```

```
    return pos;
```

```
}
```

循环n-1次

```
template<class T>
```

```
int SelectSort(T a[], int n)
```

```
{
```

```
    for (int size=n; size>1; size--)
```

```
    {
```

```
        int j=Max(a, size);
```

```
        Swap(a[j], a[size-1]);
```

```
    }
```

```
}
```

结论：比较次数 $(n-1)n/2$
移动次数 $3(n-1)$



选择排序（动画）

- https://www.bilibili.com/video/BV175411U71n?spm_id_from=333.788.videopod.sections
- https://www.bilibili.com/video/BV1DP4y1n71W/?spm_id_from=333.337.search-card.all.click



选择排序：改进思想

一般情况

| 89 45 68 90 29 34 17

次优情况

| 90 29 34 45 68 89 17

最优情况

| 17 29 34 45 68 89 90



改进选择排序

```
template<class T>
void SelectionSort(T a[], int n)
{
    bool sorted = false;
    for (int size = n; !sorted && (size > 1); size--) {
        int pos = 0;
        sorted = true;

        for (int i = 1; i < size; i++)
            if (a[pos] <= a[i]) pos = i;
        else sorted = false;
        Swap(a[pos], a[size - 1]);
    }
}
```

最好情况：只执行一次， $n-1$ 次比较

最坏情况与原来版本一样

数组有序，此语句不会被执行——
永远执行true分支——
 $a[0] \leq a[1] \leq \dots \leq a[n-1]$



冒泡排序：思想

- 首先**冒出**最小（大）元素，具体过程是：
 - 对相邻元素进行比较，如果左边的元素大于右边的元素，则交换
- 然后在余下的 $n-1$ 个元素中**冒出**最小（大）的元素；
- 如此进行下去，直至全部排完。

$$A_0 \leq A_1 \leq \cdots A_{i-1} \mid A_i, \cdots A_{j-1} \overset{?}{\leftrightarrow} A_j \cdots, A_{n-1}$$



冒泡排序： 示例

89 45 68 90 29 **34** **17**

89 45 68 90 **29** **17** 34

89 45 68 **90** **17** 29 34

89 45 **68** **17** 90 29 34

89 **45** **17** 68 90 29 34

89 **17** 45 68 90 29 34

17 | 89 45 68 90 29 34

17 | 89 45 68 **90** **29** **34**

17 | 89 45 **68** **29** 90 34

17 | 89 **45** **29** 68 90 34

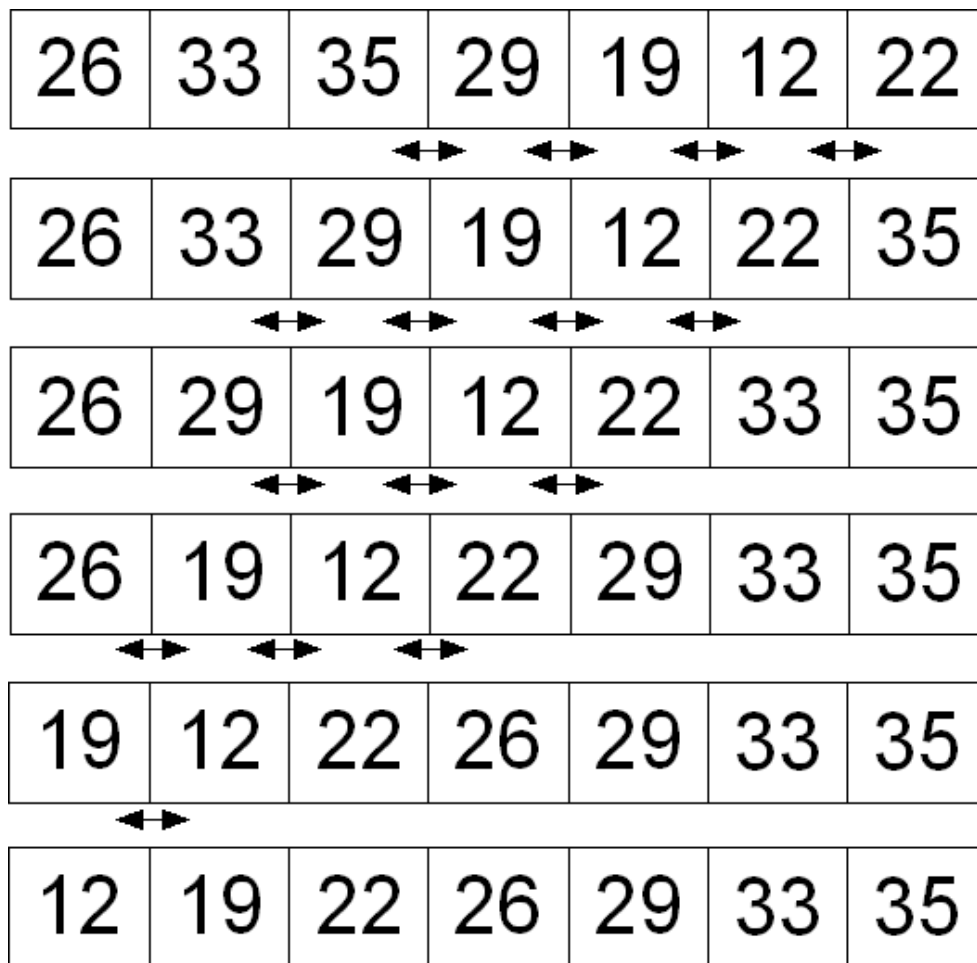
17 | **89** **29** 45 68 90 34

17 29 | 89 45 68 90 34

.....



冒泡排序：示例2



冒泡排序：代码和分析

```
template<class T>
void Bubble(T a[], int n)
{
    for (int i=0; i < n-1; i++)
    {
        if (a[i+1] < a[i])
            Swap(a[i], a[i+1]);
    }
```

```
template<class T>
void BubbleSort(T a[], int n)
{
    for (int size=n; size>1; size--)
    {
        Bubble(a, size);
    }
```

循环n-1次

结论：比较次数 $(n-1)n/2$
移动次数不确定，但应该比选择排序多



冒泡排序（动画）

https://www.bilibili.com/video/BV1f34y1s7ao/?share_source=copy_web

https://www.bilibili.com/video/BV1DP4y1n71W/?spm_id_from=333.337.search-card.all.click



改进冒泡排序

```
template<class T>
bool Bubble(T a[], int n)
{
    bool swapped = false;
    for (int i = 0; i < n - 1; i++)
        if (a[i] > a[i+1]) {
            Swap(a[i], a[i + 1]);
            swapped = true;
        }
    return swapped;
}
```

```
template<class T>
void BubbleSort(T a[], int n)
{
    for (int size = n; size > 1
        && Bubble(a, size); size--);
}
```

最好情况Bubble只执行一次， $n-1$ 次比较；最坏情况与原来版本一样

进行了相邻数据交换
如果这趟扫描没有交换呢？
所有 $a[i] \leq a[i+1]$ ——有序



插入排序：思想

- 从无序部分取一元素插入到有序部分的正确位置上，类似于玩家整理手中的扑克牌
- 把 $L[i+1]$ 插入到 $L[1 \cdots i]$ 中的正确位置， $i++$
- 直至全部插完停止


$$A_0 \leq \cdots A_j < A_{j+1} \leq \cdots A_{i-1} \mid A_i \cdots A_{n-1}$$



插入排序： 示例

89 | 45 68 90 29 34 17

45 89 | 68 90 29 34 17

45 68 89 | 90 29 34 17

45 68 89 90 | 29 34 17

45 68 89 ~~90~~ 90 34 17

45 68 ~~89~~ 89 90 34 17

45 ~~68~~ 68 89 90 34 17

~~45~~ 45 68 89 90 34 17

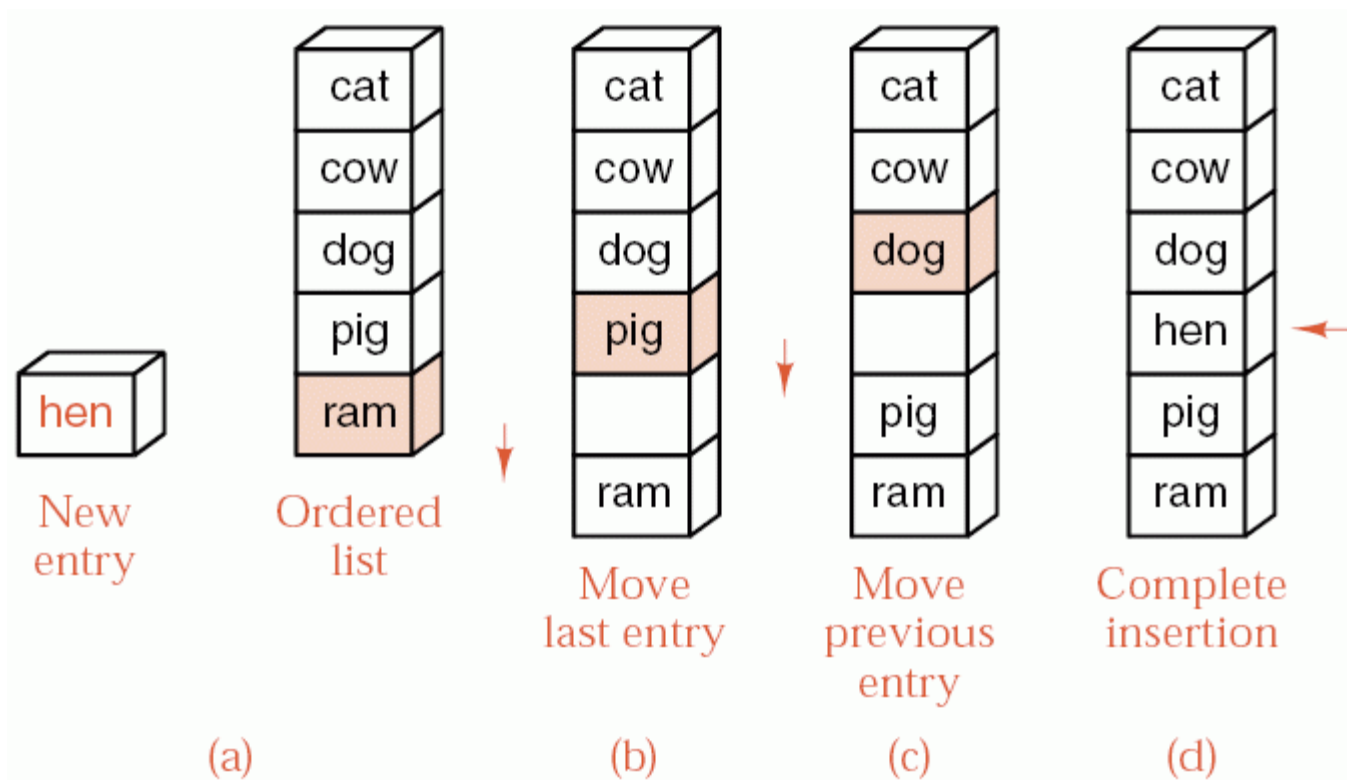
29 45 68 89 90 | 34 17

29 34 45 68 89 90 | 17

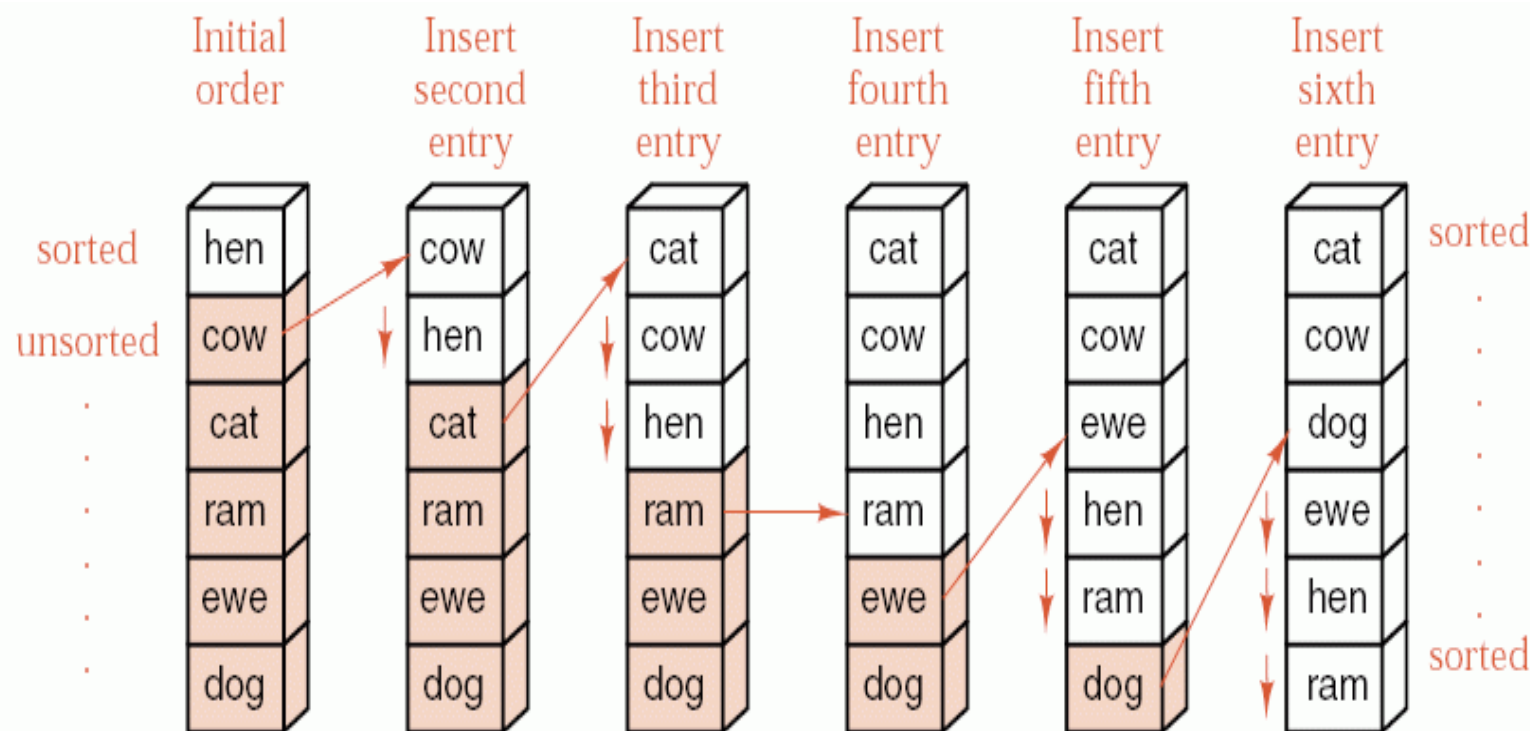
17 29 34 45 68 89 90



插入排序示例2



插入排序示例2



插入排序：代码和分析

```
template<class T>
void Insert(T a[], int n, const T& x)
{
    int i;
    for (i = n-1; i >= 0 && x < a[i]; i--)
        a[i+1] = a[i];
    a[i+1] = x;
}
```

```
template<class T>
void InsertionSort(T a[], int n)
{
    for (int s = 1; s < n; s++) {
        T next = a[s];
        Insert(a, s, next);
    }
}
```

插入数组末尾，1次比较，最好情况

插入第一个元素之前或之后，n次比较，最坏情况

平均比较次数

$(1+2+\dots+n+n)/(n+1) = n/2 + n/(n+1)$

1个元素：自然有序

Insert(a, 1, next): 2个有序

Insert(a, 2, next): 3个有序

...

Insert(a, n-1, next): 数组完全排序

结论：比较次数约为 $n^2/4$
优于选择排序



插入排序（动画）

- https://www.bilibili.com/video/BV1h5411U7wt?spm_id_from=333.788.videopod.sections
- https://www.bilibili.com/video/BV1DP4y1n71W/?spm_id_from=333.337.search-card.all.click



稳定 (stable) 排序

- 序列 $a_1, a_2, \dots, a_n$ 进行排序
- 对任意的 $i < j$, $a_i = a_j$,
若排序后 a_i, a_j 的位置 i', j' 一定满足
 $i' < j'$, 则称该排序算法为稳定的
- 否则, 为不稳定的

..... 26 26 26

..... 26 26 26

稳定的



..... 26 26 26

不稳定的



H1 小结

- 四种排序算法思想
 - 计数排序
 - 选择排序
 - 冒泡排序
 - 插入排序
- 以比较作为基本操作，如何评价算法优劣？
 - 与问题规模有关
 - 与具体输入有关（最优情况、最差情况）

